3D 게임프로그래밍 과제 3

제출일	2024. 06. 19	**전공**	게임공학과
과목	3D 게임프로그래밍	**학번**	2019182025
담당교수	이용희	**이름**	원정우

목차

1. 과제에 대한 목표 3

2. 가정 3

3. 실행결과 3

3.1 START 실행결과 장면 3

3.2 Stage 실행화면 4

4. 조작법 5

4.1 Start Scene 5

4.2 Stage Scene 5

5. 코드 6

5.1 씬 전환 6

5.2 오리 플레이어 8

5.3 오리 오브젝트 9

5.4 오리 플레이어 구현 11

5.5 오리 플레이어 회전 13

5.6 오리 플레이어 충돌 14

5.7 오리 플레이어 점프 16

1. 과제에 대한 목표

계층구조를 활용하여 게임을 구현한다. 게임은 오리의 계층구조를 만들고 중력과 충돌을 이용하여 점프로 장애물을 피하는 게임을 만든다.

2. 가정

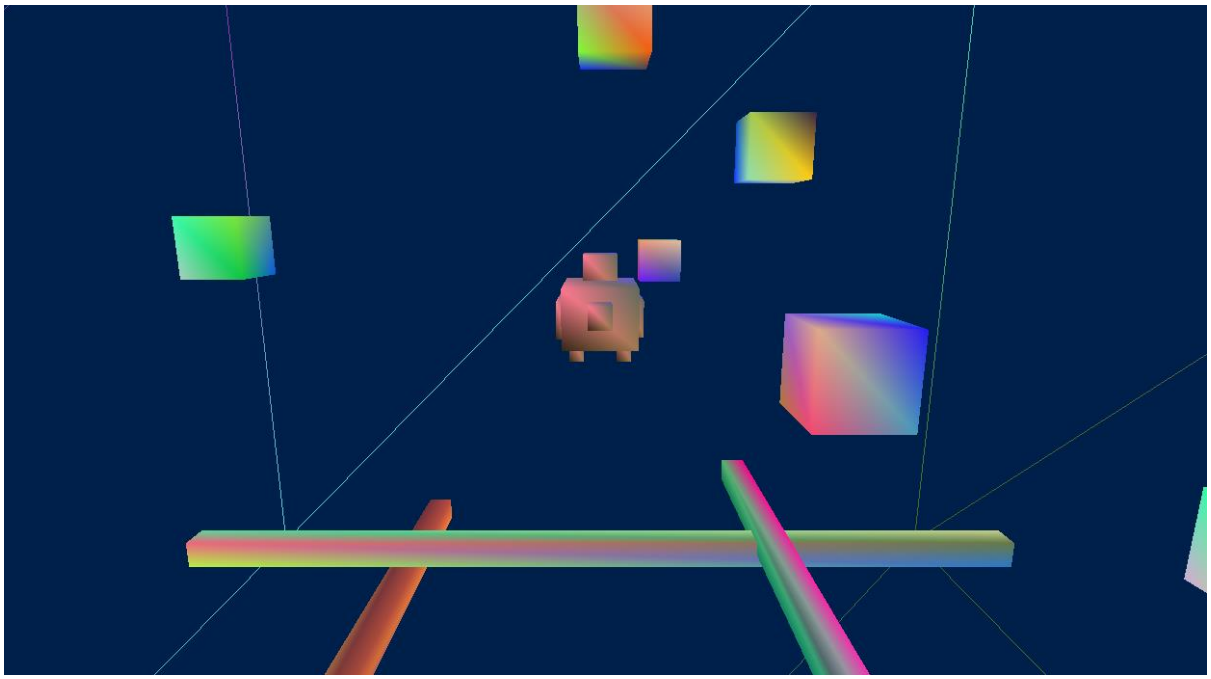
기존 과제 2를 냈던 부분에서 플레이어에 계층구조를 가지는 클래스를 이용하여 구현하면 될 것이다. 중력을 이용하여 점프하는 게임을 구현한다.

3. 실행결과

3.1 START 실행결과 장면

면으로 채워 START를 만들었고 START의 글자 중 어느 것을 눌러도 stage scene으로 넘어가게 만들었다. 플레이어는 오리의 모양을 가진다.

3.2 Stage 실행화면



또한 벽을 채워서 사용하려고 했으나 시야적으로 불편함이 있어 와이어 프레임으로 구성했다. 장애물은 바닥 근처에서 이동하는 장애물과 자유롭게 벽에 튕기며 날아다니는 장애물 2가지로 구현되어 있다. 처음에는 공중에 떠있으며 바닥으로 떨어지면서 시작한다. 죽게 되면 다시 공중에서 떨어지면서 재시작 된다. 3인칭에서 마우스를 클릭 후 회전 시 처음에는 목만 움직이다가 일정 각도 이상 회전할 경우 몸을 회전시킬 수 있다. 바닥을 제외하고 벽에 부딪히면 속도에 따라 튕기게 된다.

4. 조작법

4.1 Start Scene

키보드는 F1,3 – 1,3인칭 화면이고 각 w,a,s,d로 오리 기준 앞뒤 좌우를 이동할 수 있으며 page 업, page다운 키를 통해 오리 기준 위, 아래를 이동할 수 있다.

마우스 클릭 후 드래그 할 경우 화면의 회전을 볼 수 있다. 마우스와 방향키만으로 자유롭게 움직일 수 있게 구현했다.

4.2 Stage Scene

```
case 'X':
    m_pPlayer->m_box = false;
    break;
case 'Z':
    m_pPlayer->m_box = true;
    break;
```

기존 Start Scene에 있던 조작을 기본적으로 가지며 z로 플레이어 충돌 박스를 키고 x로 플레이어 충돌 박스를 끌 수 있다.

```
case VK_SPACE:
    if (m_pPlayer && m_pScene->type != 0) m_pPlayer->Jump();
    break;
```

스페이스바를 누르면 플레이어가 점프를 한다.

5. 코드

5.1 씬 전환

```
virtual void CheckPlayerByWallCollisions();
void CheckplayerByObjectCollisions();
virtual void CheckObjectByWallCollisions();
```

CScene은 기존 과제 2의 코드에서 충돌처리에 대한 부분이 수정되었다. 각 스테이지를 구현하기 위해서 2가지 클래스를 추가했고 state_Scene에선 벽과의 충돌을 위해 2개의 함수[1]를 구현했다.

start 신에서는 CstartShader를 사용하며

```
void CstartShader::BuildObjects(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList)
{
    CCubeMeshDiffused* pCubeMeshRight = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList,20.0f, 4.0f, 4.0f);
    CCubeMeshDiffused* pCubeMeshUp = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 4.0f, 20.0f, 4.0f);
    CCubeMeshDiffused* pCubeMesh = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 4.0f, 4.0f, 4.0f);
    CCubeMeshDiffused* pCubeMeshShort = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 10.0f, 4.0f, 4.0f);
    CCubeMeshDiffused* pCubeMeshShortUp = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 4.0f, 12.0f, 4.0f);

    m_nObjects = 17;
    m_ppObjects = new CGameObject * [m_nObjects];

    CGameObject* pRotatingObject = NULL;
    CGameObject* pExplosiveObject = new CGameObject();
    pExplosiveObject->SetMesh(pCubeMeshRight);
    pExplosiveObject->SetPosition(-50.0f, 0.0f, 80.0f);
    pExplosiveObject->SetMovingDirection(XMFLOAT3(0.0f, 0.0f, 0.0f));
    pExplosiveObject->SetMovingSpeed(0.0f);
    m_ppObjects[0] = pExplosiveObject;
```

CstartShader의 BuildObjects에서 과제 2과 같이 START의 도형배치가 이뤄진다.

```
m_pd3dGraphicsRootSignature = CreateGraphicsRootSignature(pd3dDevice);
m_nShaders = 2;
m_pShaders = new CObjectsShader * [m_nShaders];

CstageShader* stageShader = new CstageShader;
m_pShaders[0] = (CObjectsShader*)stageShader;
m_pShaders[0]->CreateShader(pd3dDevice, m_pd3dGraphicsRootSignature);
m_pShaders[0]->BuildObjects(pd3dDevice, pd3dCommandList);


CwallShader* wallShader = new CwallShader;
m_pShaders[1] = (CObjectsShader*)wallShader;
m_pShaders[1]->CreateShader(pd3dDevice, m_pd3dGraphicsRootSignature);
m_pShaders[1]->BuildObjects(pd3dDevice, pd3dCommandList);
```

stage 신에서는 CsageShader와 CwallShader를 2개를 이용한다. 여기서 m_pShaders[0]에는 장애물과 관련된 오브젝트들을 m_pShaders[1]에서는 벽을 넣기로 정했다.

[1] 벽과 충돌 함수는 이후에 충돌설명 부분에 있다.

```
std::random_device rd; // Non-deterministic random number generator
std::default_random_engine dre(rd());
std::uniform_real_distribution<float> uidSpeed{ 50.0f, 100.0f };
std::uniform_int_distribution<int> uidPosY{ -380, -350 };

std::uniform_int_distribution<int> uidPos{ -150,150 };
std::uniform_real_distribution<double> uidDir{ -1.0f,1.0f };
struct ObjectPosition {
    float posX;
    float posZ;
    bool isXAxis;
};
void CstageShader::BuildObjects(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList)
{
    m_nObjects = 10;
    m_ppObjects = new CGameObject * [m_nObjects];
    CExplosiveObject* pMovingObject = NULL;


    ObjectPosition positions[4] = {
        {380.0f, 0.0f, true},
        {-380.0f, 0.0f, true},
        {0.0f, 380.0f, false},
        {0.0f, -380.0f, false}
    };

    for (int i = 0; i < 4; ++i) { ... }
    for (int i = 4; i < 10; ++i) { ... }
    CreateShaderVariables(pd3dDevice, pd3dCommandList);
}
```

먼저 CstageShader의 BuildObjects에서는 큐브 메쉬를 이용하여 총 10개의 장애물을 생성하며 처음 4개의 장애물은 바닥의 길게 존재하는 장애물을 생성하며 각 면에서 하나씩 생성된다. 생성되는 위치에 따라 도형의 모양과 이동 방향이 정해진다. 오른쪽에 있는 값을 통해서 각 면에 해당하는 위치와 크기, 속도가 정해진다.

```
ObjectPosition positions[4] = {
    {380.0f, 0.0f, true},
    {-380.0f, 0.0f, true},
    {0.0f, 380.0f, false},
    {0.0f, -380.0f, false}
};

for (int i = 0; i < 4; ++i) {
    // Set cube dimensions
    float sizeX = 25.0f;
    float sizeY = 25.0f;
    float sizeZ = 25.0f;

    // Determine axis and position
    float posX = positions[i].posX;
    float posY = float(uidPosY(dre));
    float posZ = positions[i].posZ;
    bool isXAxis = positions[i].isXAxis;

    if (isXAxis) {
        sizeZ = 790.0f;
    }
    else {
        sizeX = 790.0f;
    }
```

나머지 6개의 장애물은 랜덤한 위치와 랜덤한 방향, 속도를 가진 큐브들을 생성한다.

CwallShader에서는 과제2와 동일하게 벽의 면을 채우지 않기 위해 D3D12_FILL_MODE_WIREFRAME을 사용하였고 벽 오브젝트 내부에서 이동하기 때문에 D3D12_CULL_MODE_NONE을 통해 배경이 은면처리 되지 않게 했다. CwallShader의 BuildObjects에서는 과제 2와 동일한 벽에서 활동 범위를 늘리기 위해 크기를 800으로 수정했다.

```
CCubeMeshDiffused* wallplane = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList,
    800.0f, 800.0f, 800.0f);
```

5.2 오리 플레이어

```
CDuckObject* m_pChildObject;

float m_fAngle = 0.0f;
float m_fWingAngle = 0.0f;
float m_fTotalAngle = 0.0f;
float m_fTotalWingAngle = 0.0f;
float m_fTotalTime = 0.0f;
float m_fTotalWingTime = 0.0f;
float m_fElapsedTimeAfterFire = 0.0f;
float m_fCoolDown = 0.25f;

LPSTR m_pstrFrameName;
```

먼저 오리 오브젝트를 저장할 m_pChildObject와 날개, 다리의 움직임을 위한 변수들을 선언했다

```
virtual CCamera* ChangeCamera(DWORD nNewCameraMode, float fTimeElapsed);

void CreateDuck(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList);
void SetFrameName(LPSTR pstrFrameName) { m_pstrFrameName = pstrFrameName; }
void SetChildObject(CDuckObject* pChildObject) { m_pChildObject = pChildObject; }

CDuckObject* FindFrame(LPSTR pstrFrameName);

virtual void Rotate(float x, float y, float z);
virtual void Animate(float fTimeElapsed);
virtual void OnPrepareRender();
virtual void Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera);
virtual void UpdateTransform();

void SetBox(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList);
void PrepareExplosion(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList
    * pd3dCommandList);
```

오리 오브젝트를 관리하고 원하는 부위의 Frame을 찾는 FindFrame 함수, 피격 범위를 정해주는 SetBox, 폭발을 관리하는 함수와 회전, 이동, 애니메이션, Render 등을 관리하는 함수들을 사용했다.

```
m_pCamera->SetTimeLag(0.0f);
m_pCamera->SetLookAt(FindFrame("Head")->GetLook());
m_pCamera->SetPosition(FindFrame("Head")->GetPosition());
m_pCamera = OnChangeCamera(FIRST_PERSON_CAMERA, nCurrentCameraMode);
m_pCamera->SetTimeLag(0.0f);
m_pCamera->SetOffset(XMFLOAT3(0.0f, 20.0f, 0.0f));
```

1인칭인 경우 카메라는 오리의 머리 위치를 기준으로 한다. 3인칭인 경우에는 과제 2와 동일하게 카메라를 적당한 위치에 놓고 플레이어 캐릭터를 쳐다본다.

5.3 오리 오브젝트

```
class CDuckObject : public CGameObject {
public:
    CDuckObject();
    virtual ~CDuckObject();

public:
    void UpdateTransform(XMFLOAT4X4* pxmf4x4Parent);

    virtual void Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera);

private:
    LPSTR m_pstrFrameName;
    XMFLOAT4X4 m_xmf4x4Transform;
    CDuckObject* m_pChildObject;
    CDuckObject* m_pSiblingObject;
```

여기서 사용된 CDuckObject는 m_pChildObject, m_pSiblingObject를 통해 계층구조를 구현할 수 있는 오브젝트 클래스이며

```
void SetFrameName(LPSTR pstrFrameName) { m_pstrFrameName = pstrFrameName; }
void SetChildObject(CDuckObject* pChildObject) { m_pChildObject = pChildObject; }
void SetSiblingObject(CDuckObject* pSiblingObject) { m_pSiblingObject = pSiblingObject; }

void SetPos(float x, float y, float z);
void SetAngle(XMFLOAT3& f3Axis, float fAngle);
void SetRotaion(XMMATRIX& xmmtRotation);
void SetScale(XMFLOAT3& f3Scaling);
void SetRUL(XMFLOAT3& f3Right, XMFLOAT3& f3Up, XMFLOAT3& f3Look);

CDuckObject* FindFrame(LPSTR pstrFrameName);
```

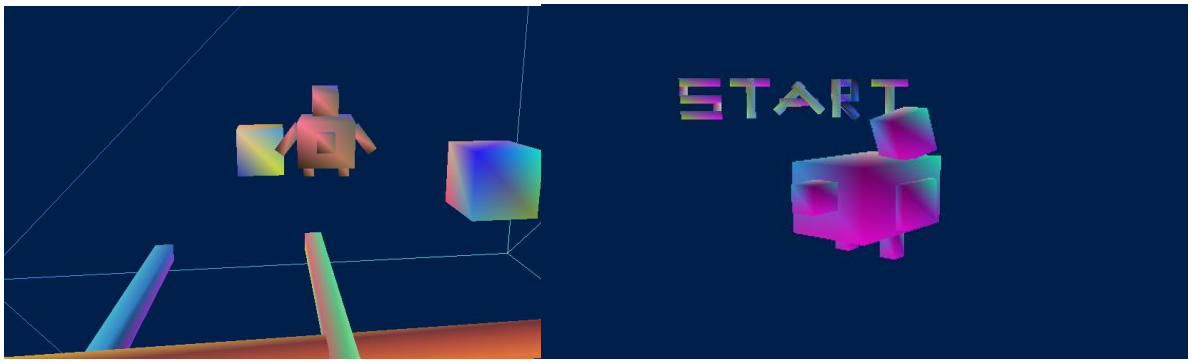
여러 Set 함수들을 통해 원하는 값을 넣어 구현할 수 있다.

```
class CPlayer : public CGameObject
{
public:
    virtual void Animate(float fElapsedTime);

    void Jump(); // 점프를 시작하는 함수
    void UpdateJump(float fTimeElapsed); // 점프 상태를 업데이트하는 함수

public:
    bool m_bIsJumping = false; // 플레이어가 점프 중인지 여부
    float m_fJumpSpeed = 600.0f; // 점프 속도
    float m_fCurrentJumpSpeed = 0.0f; // 현재 점프 속도
    float m_fJumpHeight = 0.0f; // 현재 점프 높이
    float m_fGround = 0.0f;// 현재 바닥 높이
    bool m_bGround = false;
```

추가적으로 플레이어 클래스에는 점프와 관련된 변수, 함수를 추가했다.

```
void CDuckObject::UpdateTransform(XMFLOAT4X4* pxmf4x4Parent)
{
    m_xmf4x4World = m_xmf4x4Transform;
    if (pxmf4x4Parent) m_xmf4x4World = Matrix4x4::Multiply(m_xmf4x4Transform, *pxmf4x4Parent);

    if (m_pSiblingObject) m_pSiblingObject->UpdateTransform(pxmf4x4Parent);
    if (m_pChildObject) m_pChildObject->UpdateTransform(&m_xmf4x4World);
}

void CDuckObject::Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera)
{
    CGameObject::Render(pd3dCommandList, pCamera);

    if (m_pSiblingObject) m_pSiblingObject->Render(pd3dCommandList, pCamera);
    if (m_pChildObject) m_pChildObject->Render(pd3dCommandList, pCamera);
}

CDuckObject* CDuckObject::FindFrame(LPSTR pstrFrameName)
{
    CDuckObject* pHierarchyObject = NULL;

    if (m_pstrFrameName == pstrFrameName) return this;

    if (m_pSiblingObject) if (pHierarchyObject = m_pSiblingObject->FindFrame(pstrFrameName)) return pHierarchyObject;
    if (m_pChildObject) if (pHierarchyObject = m_pChildObject->FindFrame(pstrFrameName)) return pHierarchyObject;

    return NULL;
}
```

UpdateTransform에서 계층구조를 반영하여 각 오브젝트의 월드 변환 행렬을 설정하고 업데이트하며 Render와 FindFrame에서도 계층구조를 반영하여 Render함수를 실행하거나 원하는 프레임을 찾는다.



5.4 오리 플레이어 구현

```cpp
void CDuckPlayer::CreateDuck(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList* pd3dCommandList)
{
    CMesh* pCubeMeshDiffused = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 3.0f, 3.0f, 3.0f);
    CSphereMeshDiffused* pSphereMesh = new CSphereMeshDiffused(pd3dDevice, pd3dCommandList, 8.0f, 8, 8);

    SetFrameName("duck");

    CDuckObject* pParentObject = NULL;
    CDuckObject* pChildObject = NULL;
    CDuckObject* pSiblingObject = NULL;

    pChildObject = new CDuckObject;
    pChildObject->SetPos(-10.0f, 0.0f, 0.0f);
    pChildObject->SetFrameName("LeftPelvis");
    SetChildObject(pChildObject);
    pParentObject = pChildObject;

    pChildObject = new CDuckObject;
    pChildObject->SetMesh((CMesh*)pCubeMeshDiffused);
    pChildObject->SetScale(XMFLOAT3(2.0f, 4.0f, 2.0f));
    pChildObject->SetPos(0.0f, -2.5f, 0.0f);
    pChildObject->SetFrameName("LeftLeg");
    pParentObject->SetChildObject(pChildObject);

    pSiblingObject = new CDuckObject;
    pSiblingObject->SetPos(10.0f, 0.0f, 0.0f);
    pSiblingObject->SetFrameName("RightPelvis");
    pParentObject->SetSiblingObject(pSiblingObject);
    pParentObject = pSiblingObject;

    pChildObject = new CDuckObject;
    pChildObject->SetMesh((CMesh*)pCubeMeshDiffused);
    pChildObject->SetScale(XMFLOAT3(2.0f, 4.0f, 2.0f));
    pChildObject->SetPos(0.0f, -2.5f, 0.0f);
    pChildObject->SetFrameName("RightLeg");
    pParentObject->SetChildObject(pChildObject);

    pSiblingObject = new CDuckObject;
    pSiblingObject->SetPos(0.0f, 25.0f, 0.0f);
    pSiblingObject->SetFrameName("Spine");
    pParentObject->SetSiblingObject(pSiblingObject);
    pParentObject = pSiblingObject;

    pChildObject = new CDuckObject;
    pChildObject->SetMesh((CMesh*)pCubeMeshDiffused);
    pChildObject->SetScale(XMFLOAT3(10.0f, 8.0f, 15.0f));
    pChildObject->SetPos(0.0f, -12.5f, 0.0f);
    pChildObject->SetFrameName("Back");
    pParentObject->SetChildObject(pChildObject);
    pParentObject = pChildObject;

    pSiblingObject = new CDuckObject;
    pSiblingObject->SetFrameName("Neck");
    pParentObject->SetSiblingObject(pSiblingObject);
    pParentObject = pSiblingObject;

    pChildObject = new CDuckObject;
    pChildObject->SetMesh((CMesh*)pCubeMeshDiffused);
    pChildObject->SetScale(XMFLOAT3(5.0f, 5.0f, 5.0f));
    pChildObject->SetPos(0.0f, 3.0f, 15.0f);
    pChildObject->SetFrameName("Head");
    pParentObject->SetChildObject(pChildObject);

    pSiblingObject = new CDuckObject;
    pSiblingObject->SetPos(-15.0f, -5.0f, 0.0f);
    pSiblingObject->SetFrameName("LeftWingBone");
    pParentObject->SetSiblingObject(pSiblingObject);
    pParentObject = pSiblingObject;

    pChildObject = new CDuckObject;
    pChildObject->SetMesh((CMesh*)pCubeMeshDiffused);
    pChildObject->SetScale(XMFLOAT3(2.0f, 5.0f, 10.0f));
    pChildObject->SetPos(-0, -10.0f, 5.0f);
    pChildObject->SetFrameName("LeftWing");
    pParentObject->SetChildObject(pChildObject);

    pSiblingObject = new CDuckObject;
    pSiblingObject->SetPos(15.0f, -5.0f, 0.0f);
    pSiblingObject->SetFrameName("RightWingBone");
    pParentObject->SetSiblingObject(pSiblingObject);
    pParentObject = pSiblingObject;
```

먼저 CDuckObject 생성자에서 CreateDuck을 호출하며 이 함수에서는 m_pChildObject에 LeftPelvis를 먼저 넣고 m_pChildObject, pChildObject, pSiblingObject 이 3가지를 통해 SetSiblingObject과 SetChildObject를 이용하여 계층구조를 구현한다. 관절 같은 연결 부위는 형제(sibling)로, 실제 부위(ex. 다리, 머리 등)는 각 관절에 맞춰 자식(child)으로 설정하는 방식으로 구현했다. 실제 부위에 머리, 몸, 날개, 다리, 꼬리를 만들어 오리처럼 보이도록 만들었다.

```cpp
void CDuckPlayer::Animate(float fTimeElapsed)
{
    if (m_bBlowingUp)
    {
        CPlayer::Animate(fTimeElapsed);
    }
```

먼저 Animate에서 m_bBlowingUp을 통해 터지는 상태라면 플레이어 Animate로 가서 터지는 상태를 계산한다.

```cpp
XMFLOAT3 xmf3VelocityXZ = XMFLOAT3(m_xmf3Velocity.x, 0.0f, m_xmf3Velocity.z);
if (Vector3::Length(xmf3VelocityXZ) > 50.0f) {
    m_fTotalTime += fTimeElapsed;
    m_fAngle = sinf(m_fTotalTime * 2.0f * XM_PI);
    if (m_fTotalTime > 0.5f) m_fAngle *= 2.0f;
    m_fTotalAngle += m_fAngle;
}
else {
    m_fTotalTime = 0.0f;
    m_fAngle = -0.1f * m_fTotalAngle;
    m_fTotalAngle += m_fAngle;

    if (fabs(m_fTotalAngle) < 0.01f) {
        m_fAngle = 0.0f;
        m_fTotalAngle = 0.0f;
    }
}
```

```cpp
if (m_bIsJumping) {
    m_fTotalWingTime += fTimeElapsed;
    m_fWingAngle = sinf(m_fTotalWingTime * 2.0f * XM_PI);
    if (m_fTotalWingTime > 0.25f) m_fWingAngle *= 2.0f;
    m_fTotalWingAngle += m_fWingAngle;
}
else {
    m_fTotalWingTime = 0.0f;
    m_fWingAngle = -0.1f * m_fTotalWingAngle;
    m_fTotalWingAngle += m_fWingAngle;

    if (fabs(m_fTotalWingAngle) < 0.01f) {
        m_fWingAngle = 0.0f;
        m_fTotalWingAngle = 0.0f;
    }
}
```

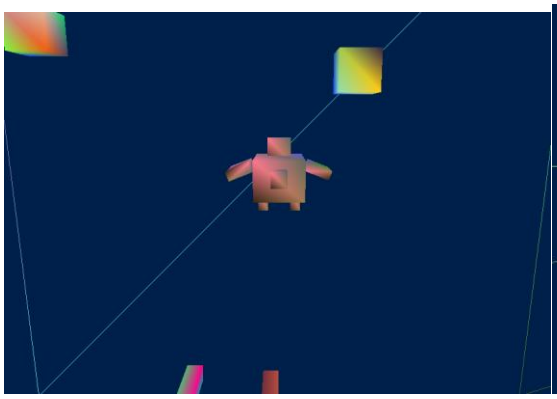
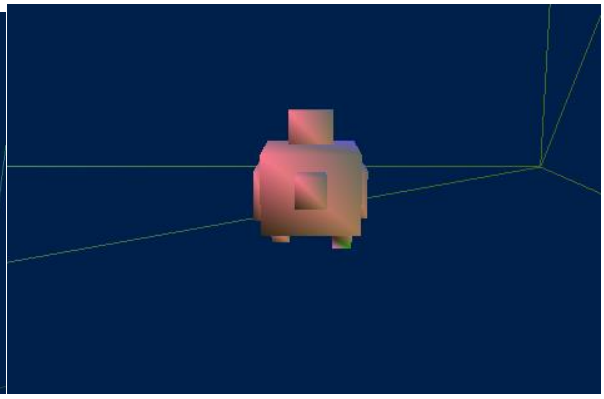
그렇지 않을 경우에는 XZ평면상에서 이동하는 속도와 점프를 하고 있는지 확인한다. 이동하는 속도가 기준을 넘어섰다면 다리를 회전하는 각도를 구한다. 각도의 주기적인 변화를 구하고 만약 기준에 부족한 상태라면 각도들을 초기화 하여 움직이기 전의 모습으로 되돌린다. 날개 또한 점프를 하고 있는 상태(m_bIsJumping가 true)라면 날개 짓을 하는 것을 구현하기 위한 각도를 구한다.

```
FindFrame("LeftWingBone")->SetAngle(XMFLOAT3(0.0f, 0.0f, 1.0f), -m_fWingAngle);
FindFrame("RightWingBone")->SetAngle(XMFLOAT3(0.0f, 0.0f, 1.0f), m_fWingAngle);

FindFrame("RightPelvis")->SetAngle(XMFLOAT3(1.0f, 0.0f, 0.0f), m_fAngle);
FindFrame("LeftPelvis")->SetAngle(XMFLOAT3(1.0f, 0.0f, 0.0f), -m_fAngle);
```

구한 두 개의 각도를 이용하여 날개와 다리에 SetAngle을 통해 넣어주어 다리가 움직이는 모습과 날개짓을 하는 모습을 만든다.

```
void CDuckPlayer::Render(ID3D12GraphicsCommandList* pd3dCommandList, CCamera* pCamera)
{
    OnPrepareRender();

    if (m_pCamera->GetMode() == THIRD_PERSON_CAMERA) {
        CPlayer::Render(pd3dCommandList, pCamera);
        if (!m_bBlowingUp) {
            if (m_pChildObject) m_pChildObject->Render(pd3dCommandList, pCamera);
        }
    }

}
```

Render에서는 m_pChildObject에 있는 오브젝트도 Render를 실행하며 CDuckObject::Render(10쪽)를 통해 계층구조에 있는 오브젝트 모두 Render를 실행한다. 현재 플레이어의 m_pChildObject는 LeftPelvis이며 관절에 해당하는 오브젝트 경우에는 Mesh가 없기 때문에 CGameObject::Render에서 Mesh가 없는 경우는 조건문을 통해 넘어가도록 수정해 두었다.

5.5 오리 플레이어 회전

```
void CDuckPlayer::Rotate(float x, float y, float z)
{
    CDuckObject* pDuckObject = FindFrame("Neck");

    // Update pitch with limits
    if (x != 0.0f) {
        m_fPitch += x;
        if (m_fPitch > 10.0f) m_fPitch = 10.0f;
        if (m_fPitch < -20.0f) m_fPitch = -20.0f;
    }

    // Update yaw and wrap around
    if (y != 0.0f) {
        m_fYaw += y;
        if (m_fYaw > 360.0f) m_fYaw -= 360.0f;
        if (m_fYaw < 0.0f) m_fYaw += 360.0f;
    }
```

먼저 오리 캐릭터의 목 오브젝트를 찾아두고 피치는 -20도에서 10도 사이로 제한하고 요는 0도에서 360도 사이에서 순환한다.

```
if (335.0f < m_fYaw || m_fYaw < 25.0f) {
    XMMATRIX rotationMatrix = XMMatrixMultiply(XMMatrixRotationX(XMConvertToRadians(m_fPitch)), XMMatrixRotationY(XMConvertToRadians(m_fYaw)));
    XMFLOAT3 xmf3Look;
    XMStoreFloat3(&xmf3Look, rotationMatrix.r[2]);

    XMFLOAT3 xmf3Right = Vector3::CrossProduct(XMFLOAT3(0.0f, 1.0f, 0.0f), xmf3Look, true);
    XMFLOAT3 xmf3Up = Vector3::CrossProduct(xmf3Look, xmf3Right, true);

    pDuckObject->SetRUL(xmf3Right, xmf3Up, xmf3Look);
}
```

m_fYaw가 335도 이상이거나 25도 이하일 때는, 피치와 요 값을 이용해 회전 행렬을 계산하고, 새로운 xmf3Right, xmf3Up, xmf3Look 벡터를 구한 뒤, 오리 목 오브젝트의 방향을 SetRUL를 이용하여 머리의 회전을 구현한다.

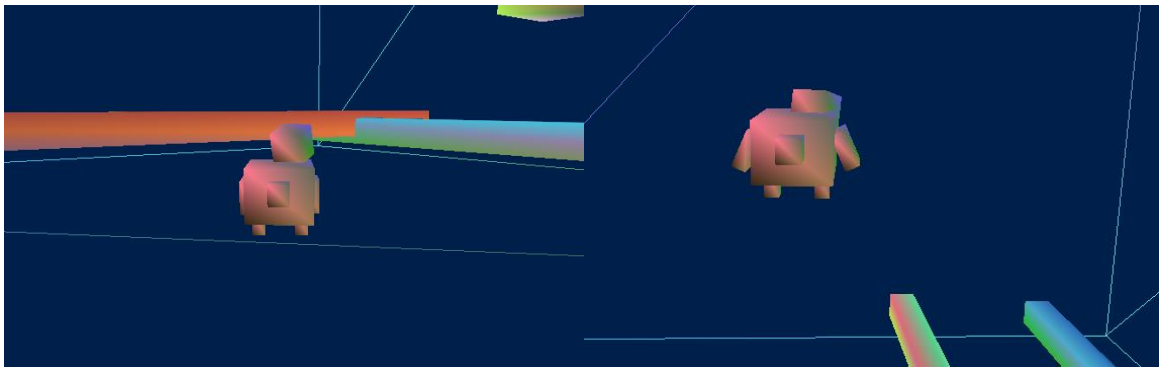
```
else {
    XMMATRIX xmmtxRotate = XMMatrixRotationAxis(XMLoadFloat3(&m_xmf3Up), XMConvertToRadians(y));
    m_xmf3Look = Vector3::TransformNormal(m_xmf3Look, xmmtxRotate);
    m_xmf3Right = Vector3::TransformNormal(m_xmf3Right, xmmtxRotate);

    pDuckObject->SetRotaion(XMMatrixRotationX(XMConvertToRadians(x)));
    m_fYaw -= y;
}
```

그렇지 않은 경우에는 주어진 y 값에 대해 오리의 회전을 계산하여 구현한다.

```
if (m_pCamera->GetMode() == FIRST_PERSON_CAMERA) {
    m_pCamera->SetLookAt(pDuckObject->GetLook(), pDuckObject->GetUp());
}
```

```
void CCamera::SetLookAt(XMFLOAT3& xmf3LookAt, XMFLOAT3& xmf3Up)
{
    m_xmf3Look = xmf3LookAt;
    m_xmf3Right = Vector3::CrossProduct(xmf3Up, m_xmf3Look);
    m_xmf3Up = Vector3::CrossProduct(m_xmf3Look, m_xmf3Right);
}
```

카메라 모드가 1인칭일 경우, 카메라의 시선 방향과 위 방향을 SetLookAt 함수를 통해 설정합니다.



5.6 오리 플레이어 충돌

```
void CDuckPlayer::SetBox(ID3D12Device* pd3dDevice, ID3D12GraphicsCommandList
    * pd3dCommandList)
{

    CCubeMeshDiffused* pCubeMesh = new CCubeMeshDiffused(pd3dDevice, pd3dCommandList, 30.0f, 30.0f, 30.0f);

    CRotatingObject* pExplosiveObject = new CRotatingObject();
    pExplosiveObject->SetMesh(pCubeMesh);
    pExplosiveObject->SetPosition(XMFLOAT3(GetPosition().x, GetPosition().y+10.0f, GetPosition().z) );
    pExplosiveObject->SetRotationAxis(XMFLOAT3(0.0f, 1.0f, 0.0f));
    pExplosiveObject->SetRotationSpeed(0.0f);
    pExplosiveObject->SetMovingDirection(XMFLOAT3(0.0f, 0.0f, 0.0f));
    pExplosiveObject->SetMovingSpeed(0.0f);
    m_pbox = pExplosiveObject;
}
```

충돌 처리에서 각 부위별로 충돌처리를 하지 않고 따로 충돌박스를 만들어 구현했다. SetBox는 CDuckPlayer 생성자에서 호출된다.

```
    m_pbox->SetPosition(XMFLOAT3(GetPosition().x, GetPosition().y + 10.0f, GetPosition().z));
    m_pbox->Animate(fTimeElapsed);
```

CDuckPlayer::Animate 함수에서 실행되며 SetPosition으로 적절한 위치로 옮겨주고 충돌처리를 위해 m_pbox->Animate도 실행한다. m_pbox->Animate를 실행하면 CGameObject::Animate에 있는 UpdateBoundingBox();를 통해 충돌박스가 업데이트된다.

```
void stage_Scene::AnimateObjects(float fTimeElapsed)
{
    CScene::AnimateObjects(fTimeElapsed);

    CheckPlayerByWallCollisions();
    CheckplayerByObjectCollisions();
    CheckObjectByWallCollisions();
}
```

충돌처리는 총 3가지가 실행되며

```
void stage_Scene::CheckPlayerByWallCollisions()
{
    if (m_nShaders > 1){
        for (int i = 0; i < m_pShaders[1]->m_nObjects; i++)
        {
            ContainmentType containType = m_pShaders[1]->m_ppObjects[i]->m_xmOOBB.Contains(m_pPlayer->m_pbox->m_xmOOBB);
```

CheckPlayerByWallCollisions에서 플레이어의 충돌 박스(m_pPlayer->m_pbox->m_xmOOBB)를 통해 충돌을 확인한다.

```
        switch (containType)
        {
        case DISJOINT:
        {
            XMFLOAT3 wallPos = m_pShaders[1]->m_ppObjects[i]->GetPosition();
            XMFLOAT3 playerPos = m_pPlayer->GetPosition();

            float& x = playerPos.x;
            float& y = playerPos.y;
            float& z = playerPos.z;

            // 절댓값이 가장 큰 값을 찾아 50.0f 또는 -50.0f로 변경하고 나머지는 0으로 설정
            if (std::abs(x) >= std::abs(y) && std::abs(x) >= std::abs(z)) {
                x = (x < 0 ? -400.0f : 400.0f); y = 0.0f; z = 0.0f;
            }
            else if (std::abs(y) >= std::abs(x) && std::abs(y) >= std::abs(z)) {
                x = 0.0f; y = (y < 0 ? -400.0f : 400.0f); z = 0.0f;
                if (y == -400.0f)
                {
                    m_pPlayer->UpdateJump(y);
                }
            }
            else {
                x = 0.0f; y = 0.0f; z = (z < 0 ? -400.0f : 400.0f);

            }
            XMFLOAT3 normalVec = Vector3::Normalize(Vector3::Subtract(wallPos, XMFLOAT3(x,y,z)));
            XMFLOAT3 playerDirection = m_pPlayer->GetVelocity();
            float dotProduct = Vector3::DotProduct(playerDirection, normalVec);
            XMFLOAT3 reflectedDirection = Vector3::Subtract(playerDirection, Vector3::ScalarProduct(normalVec, 2.0f * dotProduct));
            m_pPlayer->SetVelocity(reflectedDirection);
            break;
        }
        case INTERSECTS: { ... }
        case CONTAINS:
            break;
        }
```

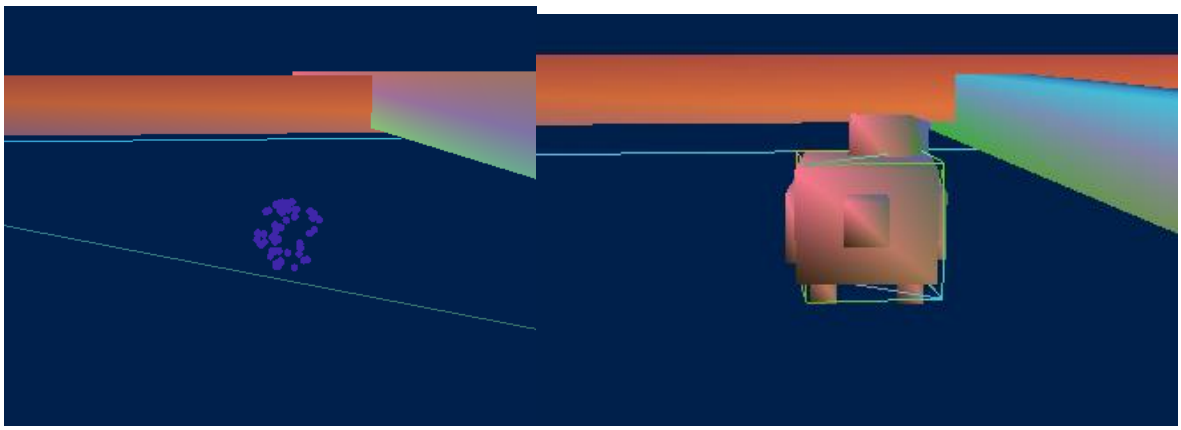
각 위치에 따른 반사 벡터를 구하며 벽 큐브의 크기가 800으로 되어있기 때문에 -400에 해당하는 벽에 닿게 되면 m_pPlayer->UpdateJump(y);[2]를 실행한다.

void stage_Scene::CheckObjectByWallCollisions()는 과제 2와 동일하기 때문에 생략한다.

```
void CScene::CheckplayerByObjectCollisions()
{
    if (m_nShaders > 1) {
        for (int i = 0; i < m_pShaders[0]->m_nObjects; i++)
        {
            if (m_pPlayer->m_pbox->m_xmOOBB.Intersects(m_pShaders[0]->m_ppObjects[i]->m_xmOOBB))
            {
                m_pPlayer->m_bBlowingUp = true;
            }
        }
    }
}
```

[2] 5.8 오리 플레이어 점프

void CScene::CheckplayerByObjectCollisions()에서도 플레이어의 충돌 박스를 이용하여 충돌처리를 구현한다.



5.7 오리 플레이어 점프

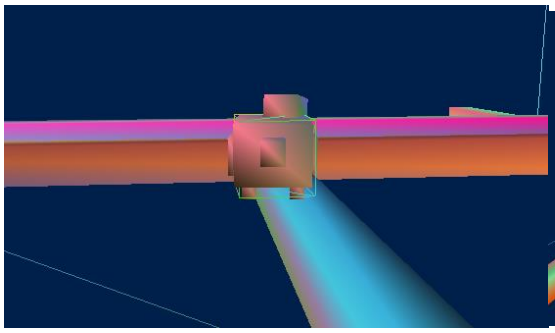
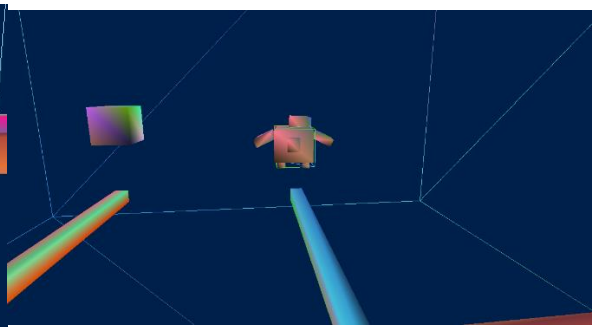
```
void CPlayer::Jump()
{
    if (!m_bIsJumping)
    {
        m_bIsJumping = true;
        m_xmf3Velocity.y = m_fJumpSpeed;
        m_xmf3Gravity.y = -600.0f;
    }
}
void CPlayer::UpdateJump(float y)
{
    m_fGround = y;
    m_bIsJumping = false;
    m_xmf3Gravity = XMFLOAT3(0, 0, 0);
    m_xmf3Velocity.y = 0.0f;
}
```

Jump 함수가 호출이 되면 점프를 하고 있는 상태가 아니면 중력과 점프 속도를 각각 넣어주고 점프상태를 true로 업데이트 합니다.

바닥에 닿는 상황이 오게 되면 바닥의 높이를 y를 통해 업데이트하고 점프 상태와 중력, 점프 속도를 false와 0으로 초기화 해둔다.

```
    m_pbox->SetPosition(XMFLOAT3(GetPosition().x, GetPosition().y + 10.0f, GetPosition().z));
    m_pbox->Animate(fTimeElapsed);
    m_xmf3Gravity = XMFLOAT3(0.0f, -600.0f, 0.0f);
```

먼저 CDuckPlayer::Animate에서 중력을 -600.0f로 설정해준다. 충돌로 인해 0으로 초기화 된 중력을 다시 만들어준다.

```
void CPlayer::Animate(float fElapsedTime)
{
    if (m_bBlowingUp)
    {
        m_fElapsedTimes += fElapsedTime;
        if (m_fElapsedTimes <= m_fDuration) { ... }
        else
        {
            m_bBlowingUp = false;
            m_bIsJumping = false;
            m_fElapsedTimes = 0.0f;
            m_xmf3Velocity = XMFLOAT3(0.0f, 0.0f, 0.0f);
            m_xmf3Position = XMFLOAT3(0.0f, 0.0f, 50.0f);
            m_xmf3Gravity = XMFLOAT3(0.0f, -600.0f, 0.0f);
        }
    }
```

터지는 상태가 끝날 때 위치와 중력 등, 점프 상태 등을 재설정한다.